KERAS - Tensor Flow

Un fond bleu et apaisant

Antony Schutz

KERAS

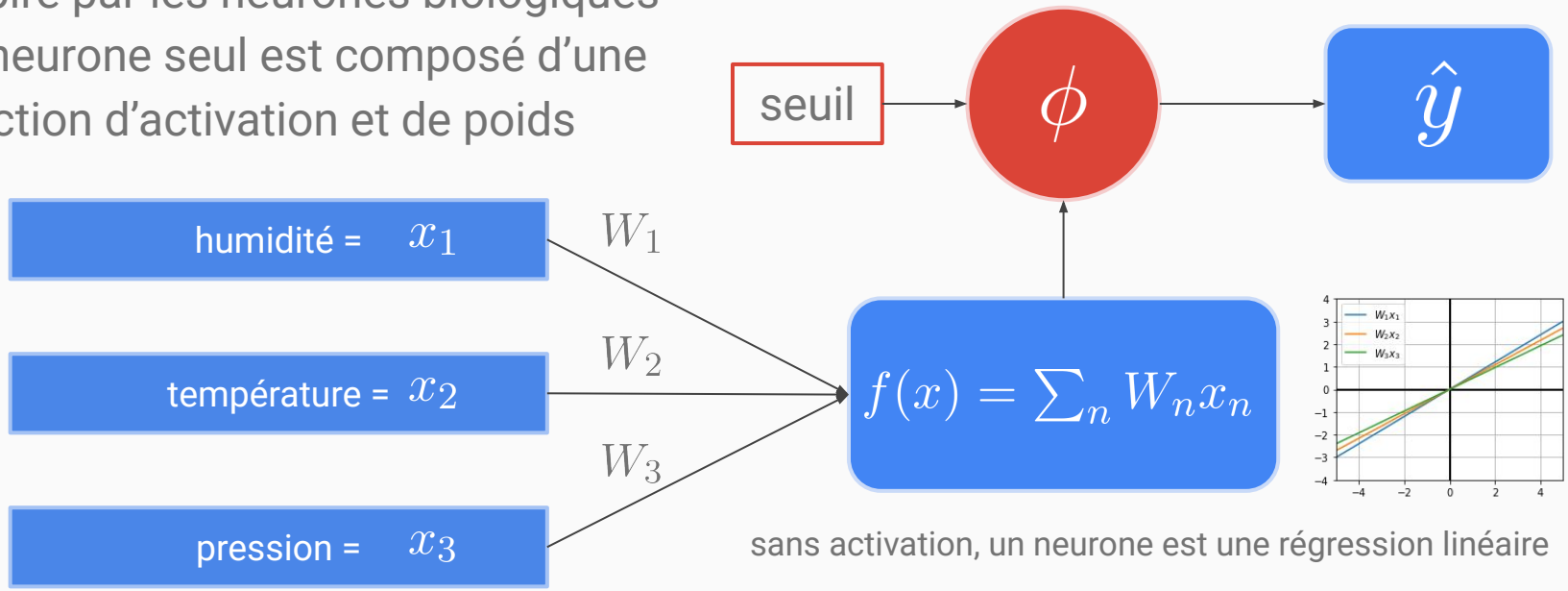
Keras est une bibliothèque open-source haut niveau en Python pour le développement d'applications d'apprentissage automatique et de deep learning. Elle offre une interface conviviale et modulaire, ce qui la rend idéale pour les débutants en apprentissage profond et les experts qui souhaitent rapidement prototyper et expérimenter.

Elle a été initialement écrite par François Chollet

https://keras.io/

Qu’est ce qu’un neurone

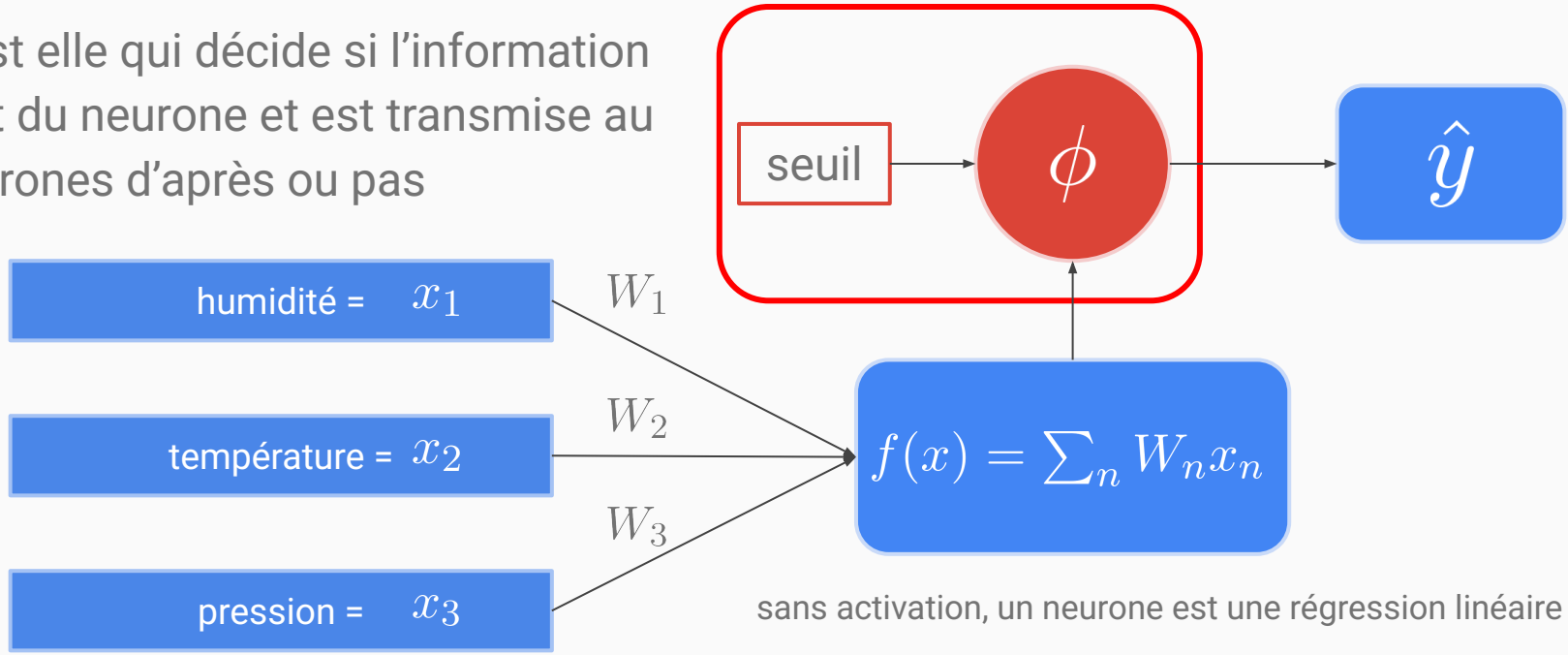
inspiré par les neurones biologiques
un neurone seul est composé d’une fonction d’activation et de poids



sans activation, un neurone est une régression linéaire

La fonction d'activation

C'est elle qui décide si l'information sort du neurone et est transmise au neurones d'après ou pas



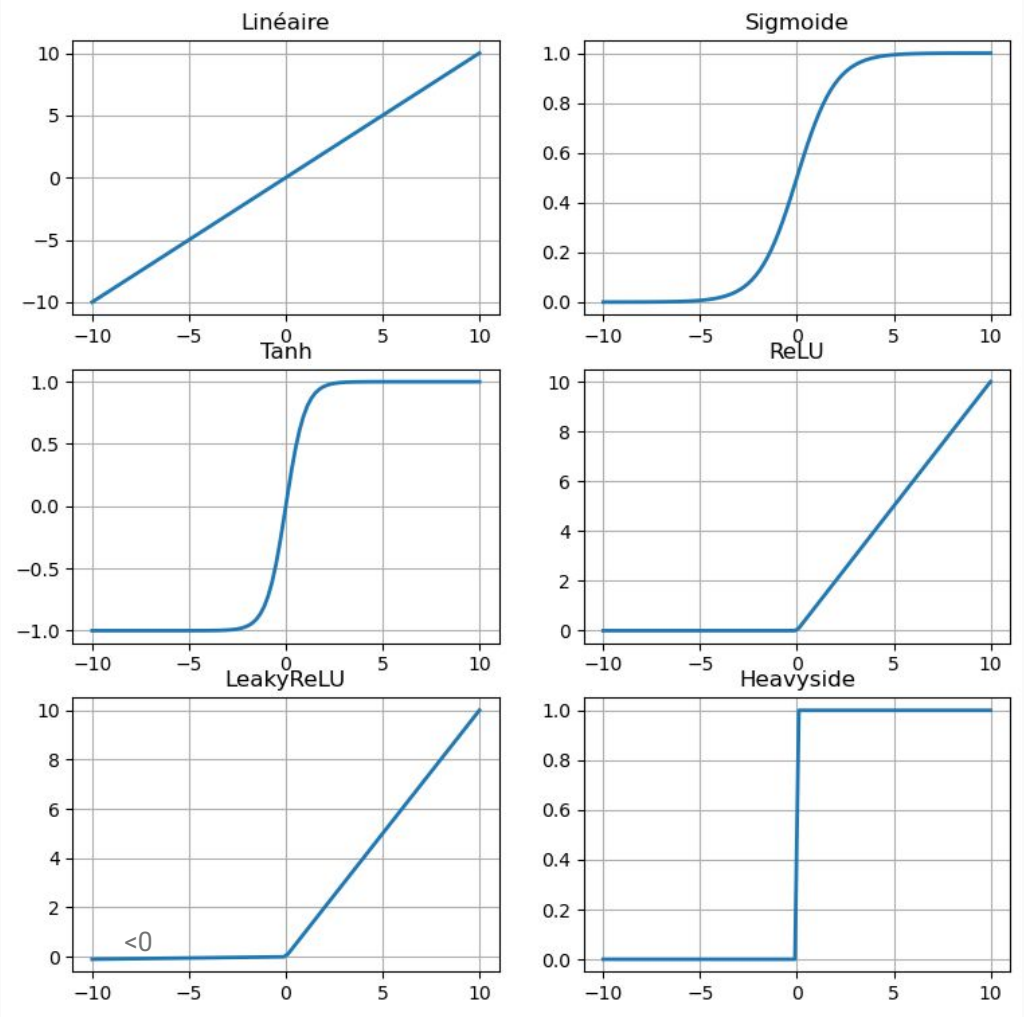
sans activation, un neurone est une régression linéaire

Fonction d’activation

La fonction d’activation est la principale source de non linéarité.

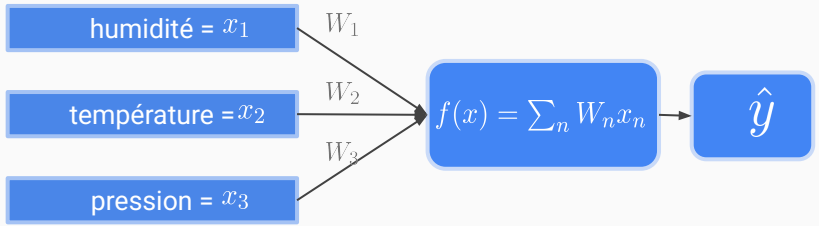
Dans un neurone biologique si le signal d’entré n’a pas assez d’énergie, il ne se passe rien

Certaine vont forcer la sortie à être entre 0 et 1 (probabilité), d’autre ne laisseront passer que les valeurs positives etc…

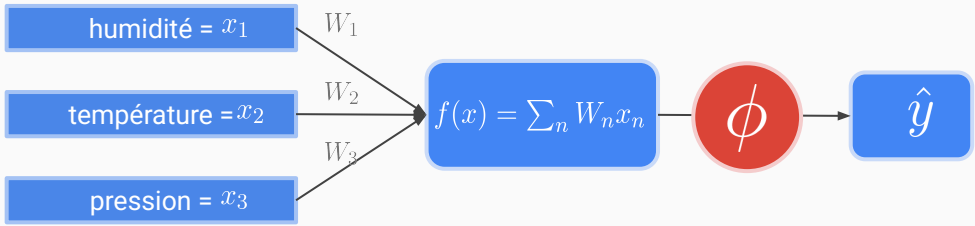


équivalence

Un neurone avec une activation linéaire est équivalent à une régression linéaire.



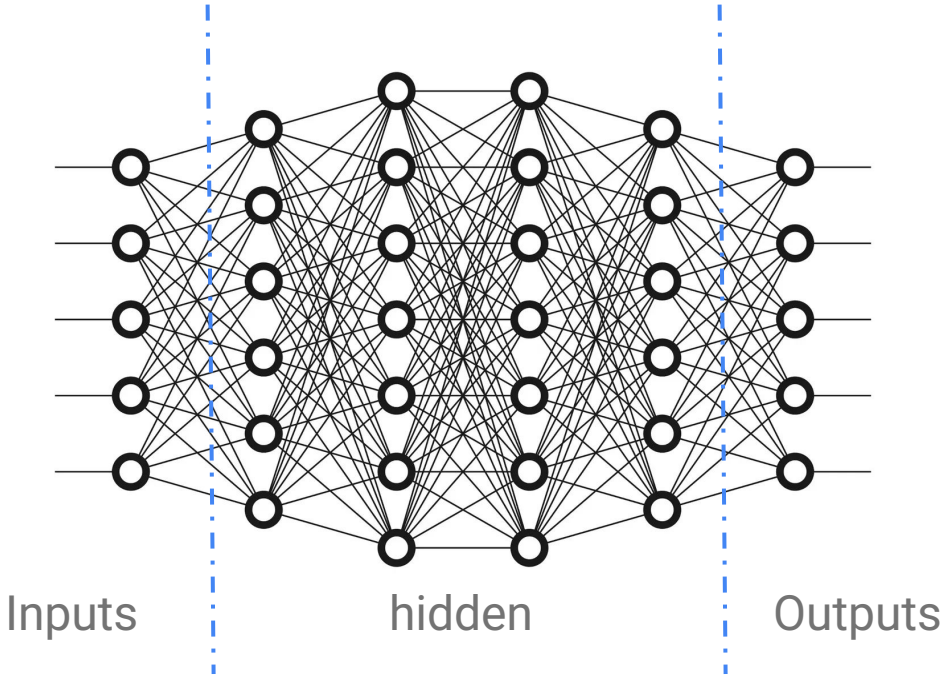
Un neurone avec une activation sigmoïde peut être équivalent à une régression logistique.



et est équivalent à une régression logistique en fonction de la **fonction de perte** utilisée.

demo

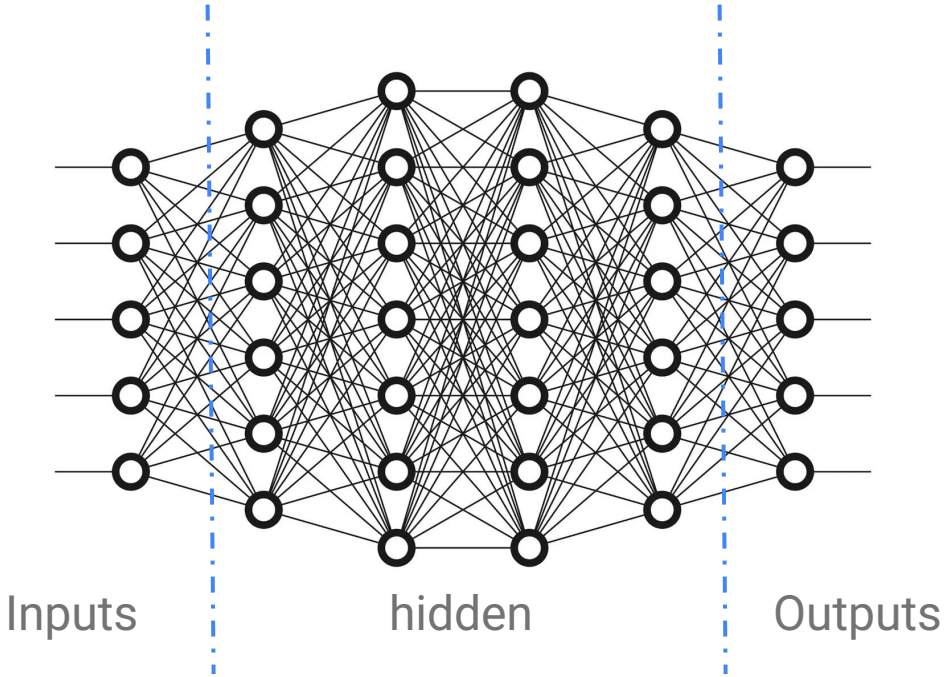
Réseau de neurones



Les réseaux de neurones sont généralement constitués:

- de couches d'entrée qui reçoivent les données à traiter
- de couches cachées qui effectuent le traitement des données
- d’une couche de sortie qui produit la sortie du réseau.

Réseau de neurones



Chaque neurone reçoit le résultat du ou des neurones qui le précède.

Ceci permet de créer des fonctions hautement non linéaire

Plus il y a de couches cachées plus le réseau est profond (deep)

Fonctions de perte

- Une fonction de perte (loss function) est une mesure utilisée pour évaluer la performance d'un modèle de réseau de neurones pendant l'entraînement.
- Elle quantifie à quel point les prédictions du modèle correspondent aux valeurs cibles (les vraies étiquettes) sur l'ensemble d'entraînement.
- L'objectif lors de l'entraînement d'un modèle est de minimiser la valeur de la fonction de perte.
- La fonction de perte est spécifiée lors de la compilation du modèle, généralement dans la méthode `compile`.
- Différentes tâches (classification, régression, etc.) et types de modèles peuvent nécessiter l'utilisation de fonctions de perte différentes.

Fonctions de perte pour la régression

Les fonctions de perte pour la régression

Les fonctions de perte pour la régression sont utilisées pour mesurer l'erreur entre les sorties du modèle et les valeurs cibles continues.

- **Mean squared error (MSE) :** Cette fonction de perte est la plus couramment utilisée pour la régression. Elle mesure la moyenne des carrés des erreurs entre les sorties du modèle et les valeurs cibles.
- **Mean absolute error (MAE) :** Cette fonction de perte mesure la moyenne des erreurs absolues entre les sorties du modèle et les valeurs cibles.
- **Huber loss :** Cette fonction de perte est une combinaison de MSE et MAE. Elle est plus robuste aux outliers que MSE.

```
model.compile(optimizer='',
loss='mean_squared_error',...)
```

```
model.compile(optimizer='',
loss='mean_absolute_error',...)
```

```
model.compile(optimizer='', loss=Huber(delta=1.0),...)
```

Fonctions de perte pour la classification

Les fonctions de perte pour la classification sont utilisées pour mesurer l'erreur entre les sorties du modèle et les classes des observations.

- **Binary cross entropy :** Cette fonction de perte est utilisée pour la classification binaire. Elle mesure la différence d'entropie entre les distributions de probabilités des sorties du modèle et des classes des observations.
- **Categorical cross entropy :** Cette fonction de perte est utilisée pour la classification multiclasse. Elle est similaire à binary cross entropy, mais elle prend en compte plusieurs classes, une seule valeur de sortie. Il vaut mieux lui donner des **y** hot encodées
- **Sparse Categorical Crossentropy :** version hot encoder naturellement

```
model.compile(optimizer='',loss='binary_crossentropy' ,...)
```

```
model.compile(optimizer='',loss='categorical_crossentropy',...
)
```

```
model.compile(optimizer='',
loss='sparse_categorical_crossentropy',...)
```

Les métriques

La principale différence entre le **loss** et la **metrics** est que le **loss** est utilisé pour minimiser l'erreur du modèle, tandis que la **metrics** est utilisée pour mesurer la performance du modèle.

Le **loss** est une fonction qui mesure la différence entre les sorties du modèle et les valeurs cibles. L'objectif de l'entraînement d'un modèle est de minimiser le **loss**. Le **loss** est généralement utilisé pour guider l'algorithme d'optimisation lors de l'entraînement du modèle.

La **metrics** est une fonction qui mesure la performance du modèle sur un ensemble de données. Les **metrics** sont utilisées pour évaluer les performances du modèle pendant l'entraînement, après l'entraînement et pour comparer les performances de différents modèles.

Les métriques

Ne change pas le comportement de l'algorithme mais permet d'évaluer les performances.

On défini la métrique utilisées dans la méthode `compile`

```
model.compile(optimizer='', loss='mean_absolute_error', metrics=['mae'])
```

On peut afficher plusieurs `metrics` en même temps

```
model.compile(optimizer=Adam(), loss='binary_crossentropy', metrics=['accuracy', 'Recall'])
```

Les métriques

Les métriques de régression

Les métriques de régression les plus couramment utilisées dans Keras sont :

- **Mean absolute error (MAE) :** Cette métrique mesure la moyenne des erreurs absolues entre les prédictions du modèle et les valeurs cibles.
- **Mean squared error (MSE) :** Cette métrique mesure la moyenne des carrés des erreurs entre les prédictions du modèle et les valeurs cibles.
- **Root mean squared error (RMSE) :** Cette métrique est la racine carrée de MSE.

Les métriques

Les métriques de classification

Les métriques de classification les plus couramment utilisées dans Keras sont :

- **Accuracy :** Cette métrique mesure le pourcentage de prédictions correctes du modèle.
- **Precision :** Cette métrique mesure le pourcentage de prédictions positives qui sont correctes.
- **Recall :** Cette métrique mesure le pourcentage de vraies positives qui sont correctement prédites.
- **F1 score :** Cette métrique est une combinaison de précision et de rappel.

Les optimiseurs

Un `optimizer` est un algorithme qui est utilisé pour minimiser la fonction de perte du modèle. L'objectif de l'entraînement d'un modèle est de minimiser la fonction de perte, ce qui permet au modèle de mieux apprendre la relation entre les entrées et les sorties des données d'entraînement.

Les `optimizers` peuvent être divisés en deux catégories principales :

- **Les optimizers stochastiques :** Ces `optimizers` utilisent un échantillon aléatoire des données d'entraînement pour mettre à jour les paramètres du modèle.
- **Les optimizers déterministes :** Ces `optimizers` utilisent toutes les données d'entraînement pour mettre à jour les paramètres du modèle.

Une pause

compile

Réseau de neurones

DEMO titanic scikeras pipeline

loss

optimizer

metrics

Le problème mathématique à résoudre

Comment on résout le problème mathématiques

La/les performance(s) que l'on affiche

Les optimiseurs stochastiques

Les `optimizers` stochastiques sont les plus couramment utilisés dans Keras. Ils sont généralement plus efficaces que les `optimizers` déterministes, car ils utilisent une plus grande quantité de données pour mettre à jour les paramètres du modèle.

Les optimizers stochastiques les plus couramment utilisés dans Keras sont :

- **Stochastic gradient descent (SGD) :** est le plus simple des `optimizers` stochastiques. Il met à jour les paramètres du modèle en suivant la direction de la pente de la fonction de perte.
- **Momentum :** utilise une quantité de momentum pour aider à la convergence. Le momentum est une valeur qui est ajoutée à la direction de la pente de la fonction de perte.
- **Adagrad :** ajuste l'échelle de la mise à jour des paramètres en fonction de la fréquence d'apparition des erreurs.
- **RMSprop :** est similaire à **Adagrad**, mais il utilise une moyenne mobile exponentielle pour calculer l'échelle de la mise à jour des paramètres.
- **Adam :** est une combinaison de **Momentum** et **RMSprop**. Il est “ considéré” comme `l'optimizer` le plus efficace.

```
model.add(Dense(num_classes, activation='softmax'))
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
```

Les optimiseurs déterministes

Les `optimizers` déterministes sont moins couramment utilisés que les optimizers stochastiques, car ils sont généralement moins efficaces. Cependant, ils peuvent être utiles dans certains cas, tels que l'entraînement de modèles sur des données sensibles ou des données avec des biais.

Les `optimizers` déterministes les plus couramment utilisés dans **Keras** sont :

- **Newton's method :** Cette méthode utilise la dérivée seconde de la fonction de perte pour calculer la mise à jour des paramètres du modèle.
- **Levenberg-Marquardt :** Cette méthode combine Newton's method avec une régularisation pour améliorer la convergence.

Ajuster le modèle

Tout comme dans sklearn ajuster le modèle se fait avec la méthode `fit` et la prédiction avec la méthode `predict`.

```
# Diviser les données en ensembles d'entraînement et de test
X train, X test, y train, y test = train test split(X, y, test size=  0.2, random state= 42)
X train, X valid, y_train, y_valid= train_test_split(X_train, y_train, test_size=  0.2, random_state= 42)
# Créer le modèle
model = Sequential()
model.add(Dense(units= 16, input dim= 10, activation= 'relu'))
model.add(Dense(units= 1, activation= 'sigmoid'))

# Compiler le modèle
model.compile(optimizer=Adam(), loss= 'binary_crossentropy' , metrics=['accuracy', 'Recall'])

# Entraîner le modèle
model.fit(X_train, y_train, epochs= 10, batch_size= 32, validation_data=(X_valid, y_valid))

# Faire des prédictions sur l'ensemble de test
y_pred = (model.predict(X_test) >  0.5).astype(int)
```

Ajuster le modèle

Tout comme dans sklearn ajuster le modèle se fait avec la méthode **`fit`** et la prédiction avec la méthode **`predict`**.

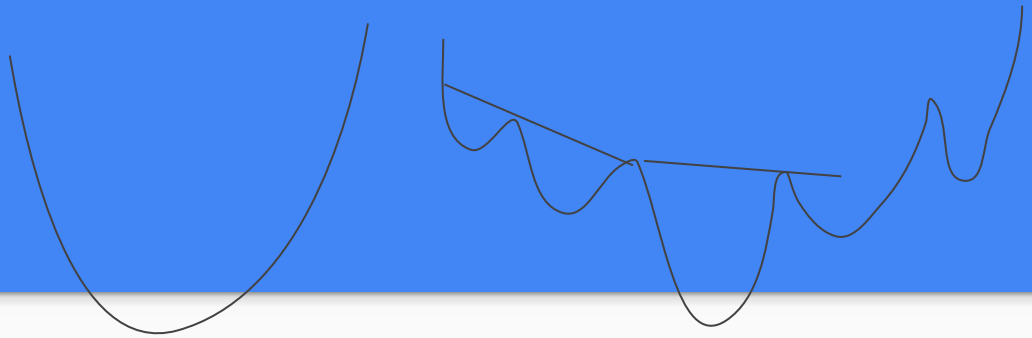
```
# Diviser les données en ensembles d'entraînement et de test
X train, X test, y train, y test = train test split(X, y, test size=  0.2, random state= 42)
X train, X valid, y_train, y_valid= train_test_split(X_train, y_train, test_size=  0.2, random_state= 42)
# Créer le modèle
model = Sequential()
model.add(Dense(units= 16, input dim= 10, activation= 'relu'))
model.add(Dense(units= 1, activation= 'sigmoid'))

# Compiler le modèle
model.compile(optimizer=Adam(), loss= 'binary_crossentropy' , metrics=['accuracy', 'Recall'])

# Entraîner le modèle
model.fit(X_train, y_train, epochs= 10, batch_size= 32, validation_data=(X_valid, y_valid))

# Faire des prédictions sur l'ensemble de test
y_pred = (model.predict(X_test) >  0.5).astype(int)
```

Ajuster le modèle



```
model.compile(optimizer=Adam(), loss= 'binary_crossentropy' , metrics=['accuracy', 'Recall'])
```

Sans surprise et sinon ca serait trop facile, les `optimizer` ont aussi des paramètres, mais on peut utiliser les paramètres par défaut.

Le learning_rate est souvent le seul

```
tf.keras.optimizers.Adam(
    learning_rate=0.001,
    beta_1=0.9,
    beta_2=0.999,
    epsilon=1e-07,
    amsgrad=False,
    weight_decay=None,
    clipnorm=None,
    clipvalue=None,
    global_clipnorm=None,
    use_ema=False,
    ema_momentum=0.99,
    ema_overwrite_frequency=None,
    jit_compile=True,
    name="Adam",
    **kwargs
)
```

Ajuster le modèle

Tout comme dans sklearn ajuster le modèle se fait avec la méthode **`fit`**

```
model.fit(X_train, y_train, epochs= 10, batch_size= 32, validation_data=(X_valid, y_valid))
```

On peut voir les métriques s'afficher au fur et à mesure sur les données d'entrainements et de validations, cela permet de voir immédiatement les risques de sur ajustement → over fitting

```
model.fit(X_train, y_train, epochs=5, batch_size=64, validation_split=0.2)
```

Ajuster le modèle

Tout comme dans sklearn ajuster le modèle se fait avec la méthode **`fit`**.

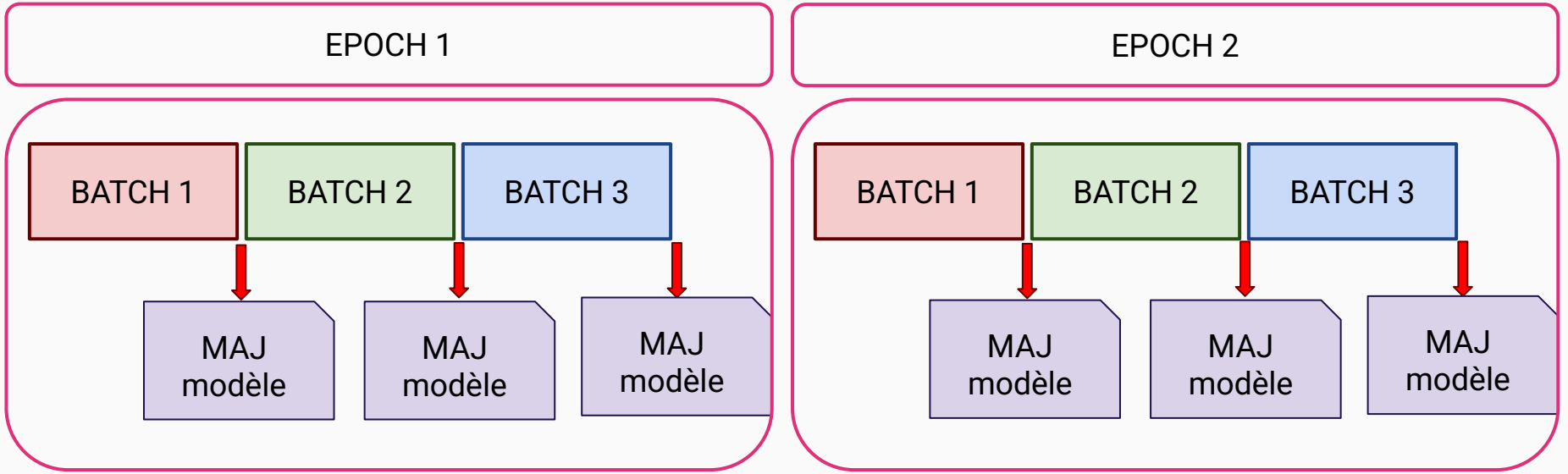
```
# Entraîner le modèle
model.fit(X_train, y_train, epochs= 10, batch_size= 32, validation_data=(X_test, y_test))
```

Le paramètre **`batch_size`** dans l'entraînement de modèles en machine learning représente **le nombre d'échantillons d'entraînement** utilisés dans une itération.

En d'autres termes, c'est le nombre d'exemples de données que le modèle va traiter à chaque **mise à jour des poids du réseau** pendant une époque.

Le paramètre **`epochs`** défini le nombre d'époque ce qui correspond à **combien de fois on parcours l'ensemble des batchs** et donc l'ensemble des données.

batch, epochs



Conseils

Quelques conseils pour définir les batch et les epochs :

- Pour les petits ensembles de données, utilisez des `batch` plus petits, tels que 8 ou 32.
- Pour les grands ensembles de données, utilisez des `batch` plus grands, tels que 32 ou 64.
- Commencez avec un nombre `d'epochs` faible, tel que 10 ou 20.
- Augmentez le nombre d'epochs progressivement jusqu'à ce que les performances du modèle ne s'améliorent plus (vous pouvez reprendre à chaud!).

En conclusion, les `batch` et les `epochs` sont des paramètres importants qui peuvent affecter les performances d'un modèle. Il est important de les définir en fonction de la taille de l'ensemble de données d'entraînement et de la complexité du modèle.

Activation de la couche de sortie

Sigmoid

1. Classification Binaire
2. Multi label : Chien, grand en même temps
   On peut appartenir à deux classes en même temps

```
model = Sequential([
    Input(shape=(64,)),
    Dense(10, activation="relu"),
    Dense(10, activation="sigmoid")
])
```

Softmax

Classification Multi-Class
Il y a plus que deux classes

```
model = models.Sequential([
    layers.Input(shape=(784,)),
    layers.Dense(10,
activation='softmax')
])
```

Créer un réseau: deux approches

Le modèle séquentiel est plus simple et plus intuitif à utiliser. Si votre architecture suit une séquence linéaire de couches (de l'entrée à la sortie), le modèle séquentiel est souvent suffisant.

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

model = Sequential()
model.add(Dense(units=64, activation='relu', input_dim=100))
model.add(Dense(units=10, activation='softmax'))
```

Si votre réseau a une architecture plus complexe, avec des branches, des partages de couches, ou des connexions non linéaires, **le modèle fonctionnel** offre une flexibilité accrue.

```
from tensorflow.keras.layers import Input, Dense
from tensorflow.keras.models import Model


input_layer = Input(shape=(100,))
hidden_layer = Dense(units=64, activation='relu')(input_layer)
output_layer = Dense(units=10, activation='softmax')(hidden_layer)


model = Model(inputs=input_layer, outputs=output_layer)
```

Créer un réseau: deux approches

Le modèle séquentiel est plus simple et plus intuitif à utiliser. Si votre architecture suit une séquence linéaire de couches (de l'entrée à la sortie), le modèle séquentiel est souvent suffisant.

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

model = Sequential()
model.add(Dense(units=64, activation='relu', input_dim=100))
model.add(Dense(units=10, activation='softmax'))
```

Nombre de neurones dans cette couche

il y a 100 features (colonnes)

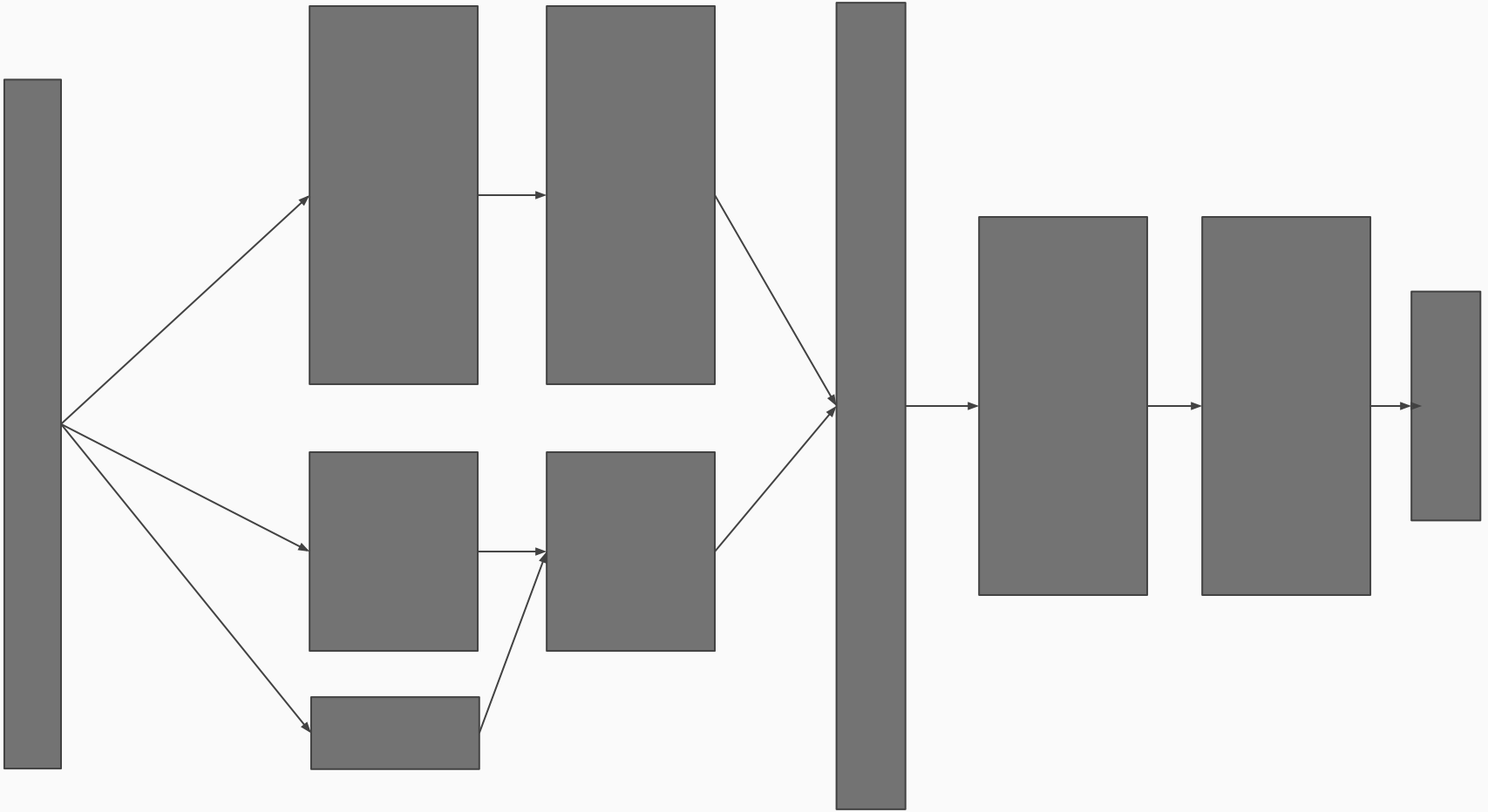
Si votre réseau a une architecture plus complexe, avec des branches, des partages de couches, ou des connexions non linéaires, **le modèle fonctionnel** offre une flexibilité accrue.

```
from tensorflow.keras.layers import Input, Dense
from tensorflow.keras.models import Model


input_layer = Input(shape=(100,))
hidden_layer = Dense(units=64, activation='relu')(input_layer)
output_layer = Dense(units=10, activation='softmax')(hidden_layer)

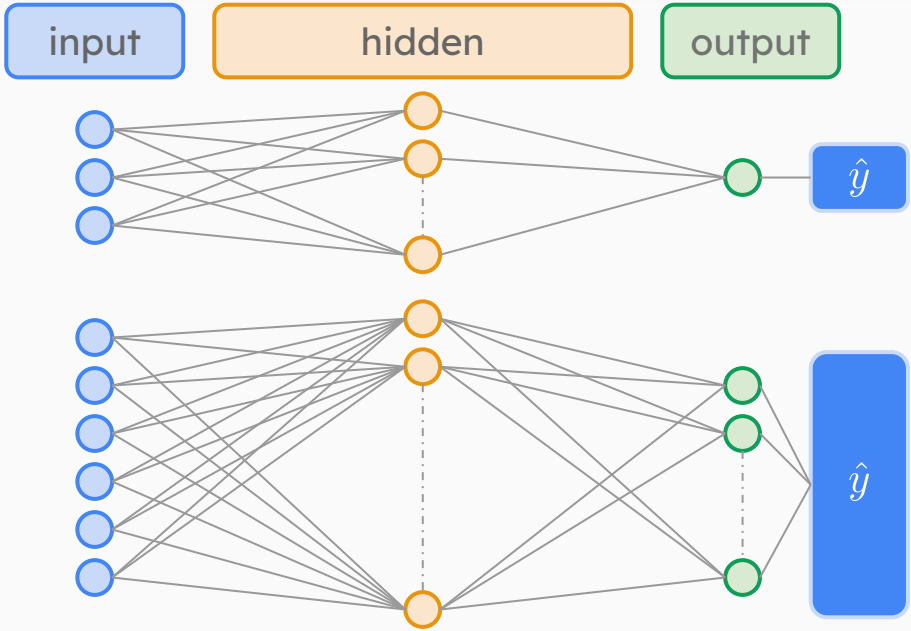

model = Model(inputs=input_layer, outputs=output_layer)
```

on peut avoir des tenseurs → des tableaux a plusieurs dimensions



Créer un régresseur/classifieur

Si on veut prédire par régression un seul paramètre ou classifier 2 classes (binaire) le réseau se terminera par un seul neurone.

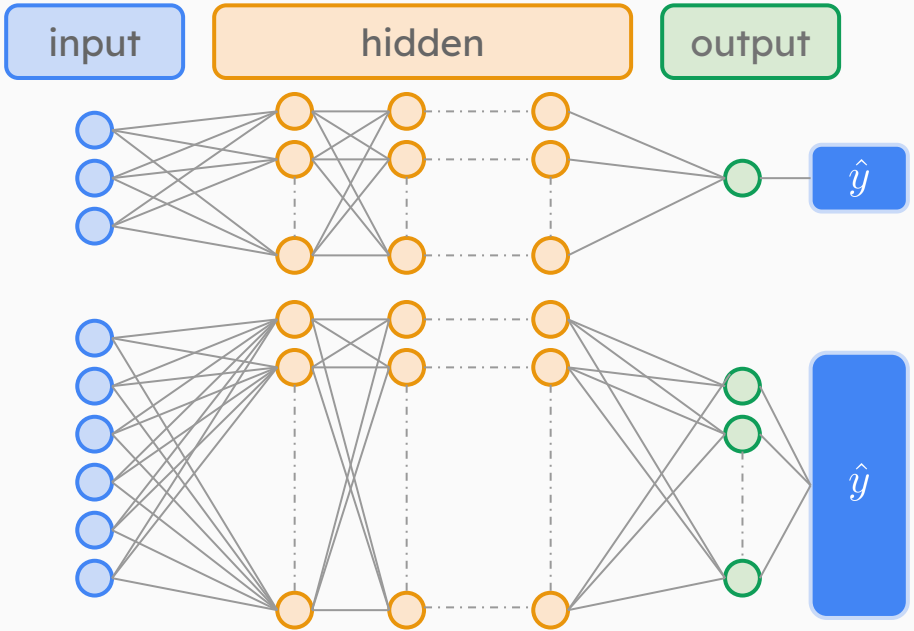


Si on veut prédire par régression plusieurs paramètres ou classifier plusieurs classes, il faudra autant de neurone que de paramètres, ou de classes, à prédire.

Attention en classification il faudra soit prendre la probabilité > 0.5 ou plutôt l'argument maximum

```
y_pred_keras = np.argmax(model_keras.predict(X_test), axis=-1)
```

Multi layer perceptron (MLP)



Les types de couches

Couches

Flatten: met à plat (vecteur) les données

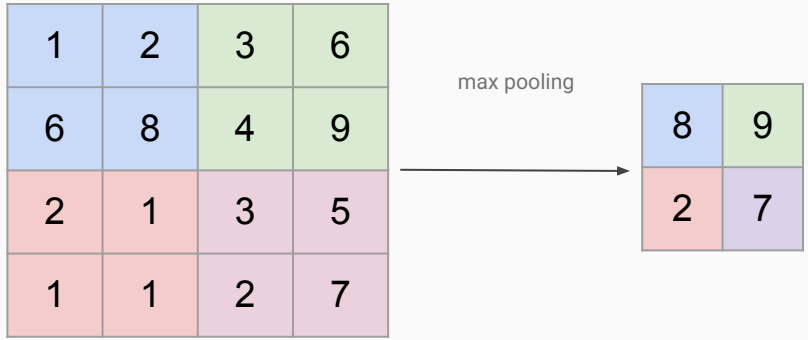
Dense: neurones classique

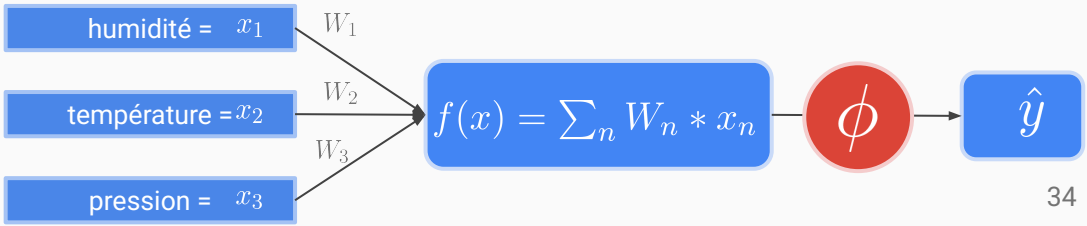
Pooling: réduit les dimensions en prenant le max, moyenne par zone, souvent utilisé après une convolution

Convolution: transforme la simple multiplication du dense par une convolution

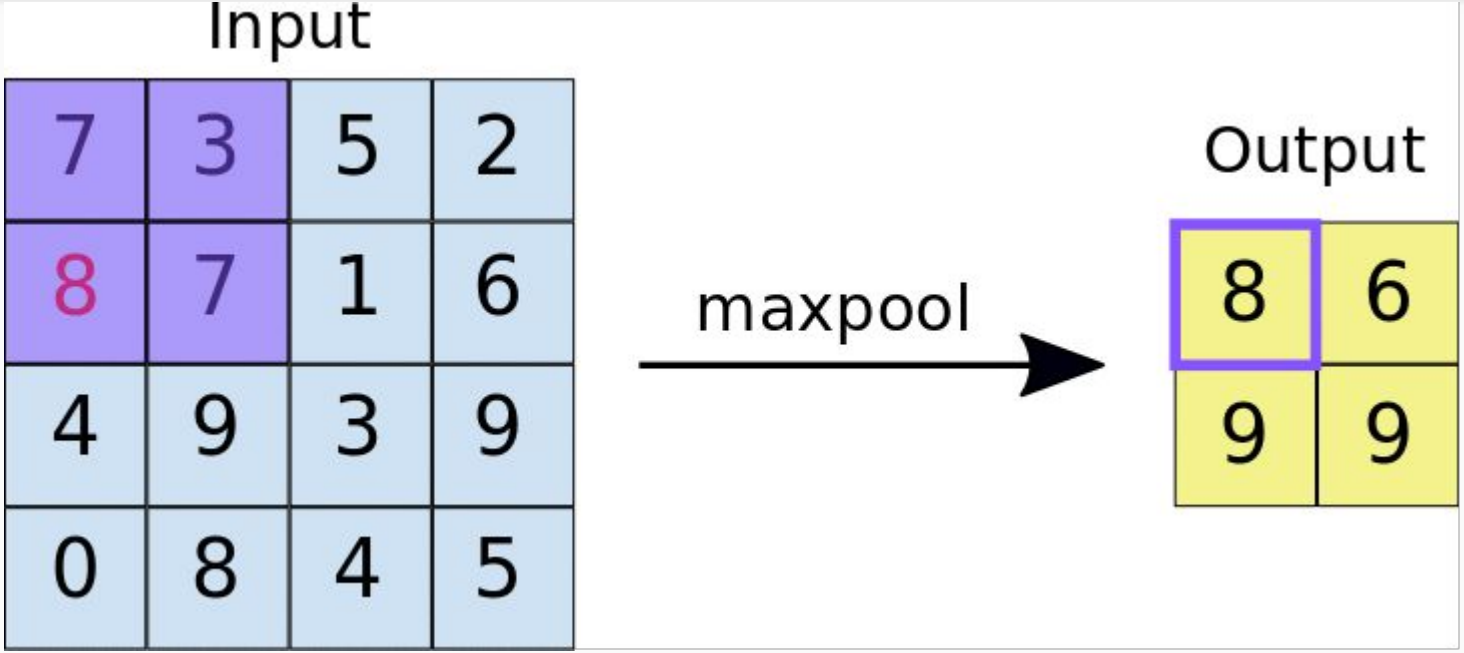
→ nécessite 1 dimension supplémentaire

```
x = np.expand_dims(x, -1)
```





Max Pooling

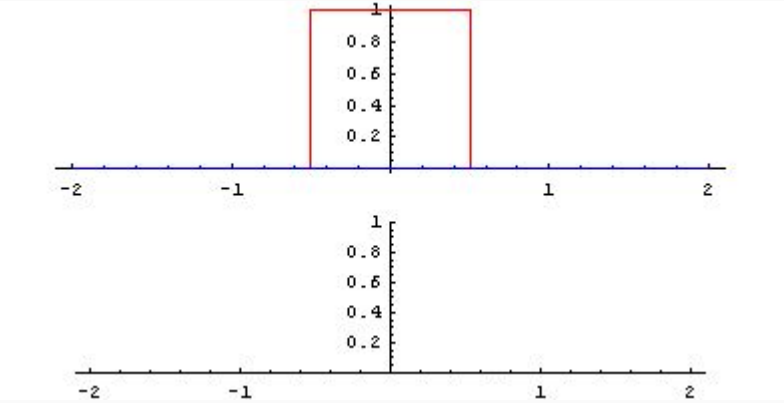


La convolution

$$(f * g)(x) = \int_{-\infty}^{+\infty} f(x-t)g(t)\,\mathrm{d}t = \int_{-\infty}^{+\infty} f(t)g(x-t)\,\mathrm{d}t$$

$$(f * g)(n) = \sum_{m=-\infty}^{\infty} f(n-m)g(m) = \sum_{m=-\infty}^{\infty} f(m)g(n-m)$$

La convolution est opérateur mathématiques comme la * / + - sauf que ca ne s'applique qu'à des fonctions



demo avec Lena filtre dérivateur

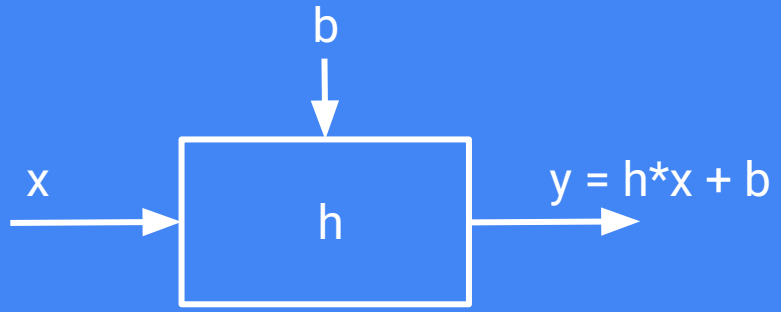
```
low_pass_filter = [[1,1,1],[1,2,1],[1,1,1]]
```

```
high_pass_filter = [[0,-1,0],[-1,5,-1],[0,-1,0]]
```

```
kernel_laplacian_5x5 = np.array([
    [-1, -1, -1, -1, -1],
    [-1, -1, -1, -1, -1],
    [-1, -1, 24, -1, -1],
    [-1, -1, -1, -1, -1],
    [-1, -1, -1, -1, -1]
])
```

Théorie des systèmes

Convolution (rappel)




```
model_conv.add( Conv2D(filters=16, kernel_size = (3,3), activation="relu") )
```

Tous les filtres:

- électronique, micro, caméra
- cavité vocale, oreille, oeil
- loupe, télescope, microscope
- ...

effectuent une opération de convolution

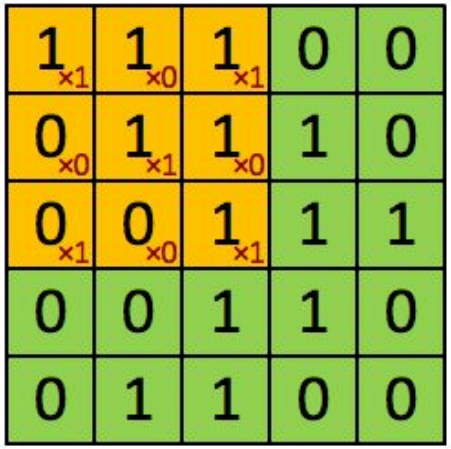
en fonction du filtre, les coefficients du filtre, le résultat sera différent:

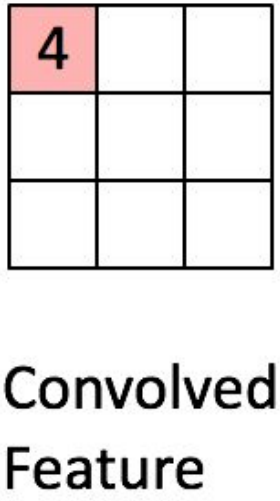
- flou sur une image, détection de contour
- filtrage passe bas ou haut en audio
- filtrage de couleur si on est en 3D

le réseau va chercher les coefficients de ces filtres pour optimiser le résultat

Convolution par un masque

1	0	1
0	1	0
1	0	1



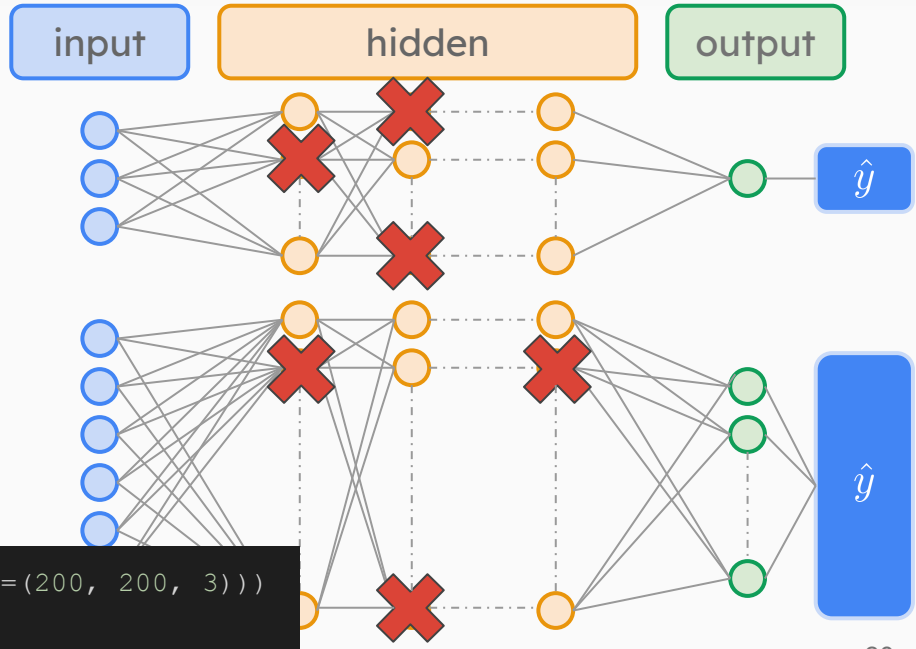


Dans un CNN, l'algorithme trouve le masque!

Régularisation

Le **dropout** est une technique de régularisation largement utilisée dans les réseaux de neurones pour prévenir le surajustement (Geoffrey Hinton 2012).

Désactive, uniquement pendant l'entraînement, un ensemble, aléatoire de neurones d'une couche.




```
model.add(Conv2D(32, (3, 3), activation='relu',input_shape=(200, 200, 3)))
model.add(MaxPooling2D((2, 2)))
model.add(Dropout(0.2))
```

Affichage …

```
Model: "sequential_9"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 flatten_9 (Flatten)         (None, 784)               0

 dense_29 (Dense)            (None, 32)                25120

 dropout_4 (Dropout)         (None, 32)                0

 dense_30 (Dense)            (None, 32)                1056

 dropout_5 (Dropout)         (None, 32)                0

 dense_31 (Dense)            (None, 10)                330

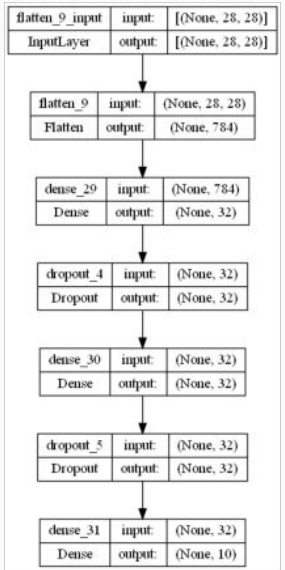
=================================================================
Total params: 26506 (103.54 KB)
Trainable params: 26506 (103.54 KB)
Non-trainable params: 0 (0.00 Byte)
_________________________________________________________________
```

```
model.summary()
```

Affiche un résumé de votre réseau et surtout le nombre total de paramètres, ici 26 506

Si on a moins de données que le nombre de paramètre les choses ne se passeront, en général, pas trop bien

Affichage …



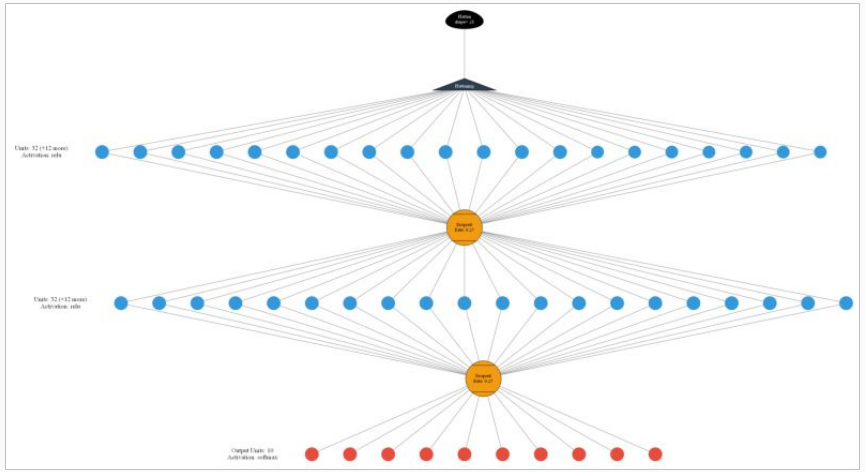

```
from tensorflow.keras.utils import plot_model

plot_model(
    model,
    to_file="model.png",
    show_shapes=True,
    show_dtype=False,
    show_layer_names=True,
    rankdir="TB",
    expand_nested=False,
    dpi=96,
    layer_range=None,
    show_layer_activations=False,
    show_trainable=False,
)

im = plt.imread("model.png")
plt.figure(figsize=(16,8))
plt.imshow(im)
plt.axis('off')
```

une autre manière d'afficher, peut être sympas dans les rapports et les présentations

Affichage …



```
from keras_visualizer import visualizer


visualizer(model,
           file_name='graph',
           file_format='png',
           settings={'MAX_NEURONS': 20})

im = plt.imread("graph.png")

plt.figure(figsize=(16,8))
plt.imshow(im)
plt.axis('off')
```

https://pypi.org/project/keras-visualizer/

Observer un neurone d’une couche

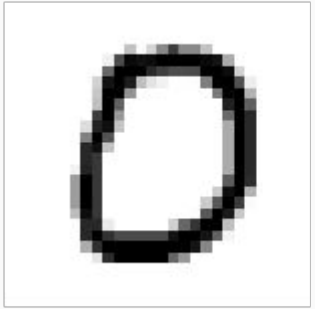
```
from tensorflow.keras.models import Model

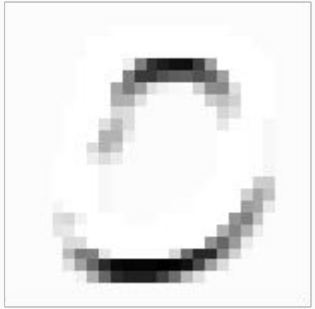
n = 10

# Sélectionner un exemple d'image du jeu de test
example_image = x_test[n].reshape(1, 28, 28, 1)

# Extraire les activations de la première couche convolutive
activation_model = Model(inputs=model.input, outputs=model.layers[0].output)
activations = activation_model.predict(example_image)

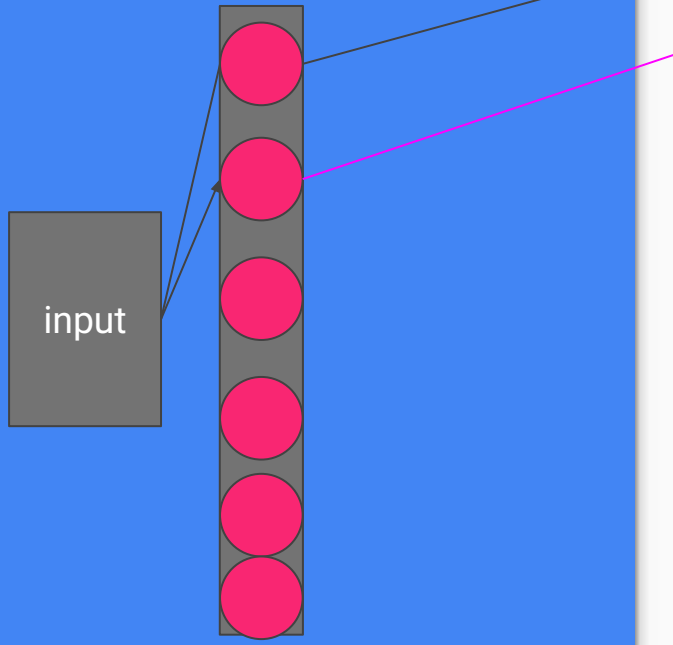
# Visualiser les activations avec une carte de chaleur
plt.subplot(121)
plt.imshow(x_test[n],cmap="gray_r")
plt.axis(False)
plt.subplot(122)
plt.imshow(activations[0, :, :, 0], cmap='gray_r')
plt.axis(False)
plt.title('Activation Map for First Convolutional Layer)'
plt.show()
```

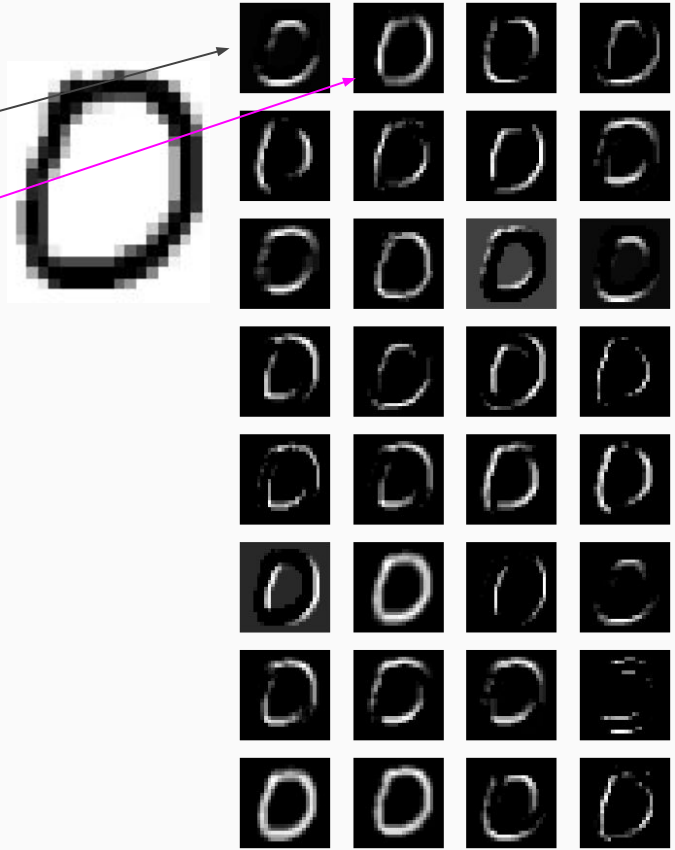




Observer tous les neurones d'une couche

```
plt.figure(figsize=(8,16))
for ind in range(32):
    plt.subplot(8,4,ind+1)
    plt.imshow(activations[0, :, :, ind], cmap='gray')
    plt.axis(False)
```





Callbacks dans Keras

Un callback est une fonction (ou un ensemble de fonctions) qui est exécutée à des moments précis pendant l'entraînement d'un modèle Keras.

Il permet de surveiller l'entraînement, d'enregistrer des informations, de modifier le comportement du modèle, et d'interagir avec le processus d'apprentissage.

ModelCheckpoint : Enregistrer le modèle à intervalles réguliers ou lorsque les performances s'améliorent.

EarlyStopping : Arrêter l'entraînement prématurément lorsque les performances du modèle cessent de s'améliorer.

TensorBoard : Visualiser les métriques d'entraînement et les graphes du modèle dans TensorBoard.

Arrêt automatique (EarlyStopping)

Sur-apprentissage : Lorsque le modèle apprend trop bien les données d'entraînement, il peut ne pas généraliser correctement aux nouvelles données.

EarlyStopping : Ce callback permet d'arrêter l'entraînement lorsque les performances du modèle sur un ensemble de validation cessent de s'améliorer.

Paramètres importants :

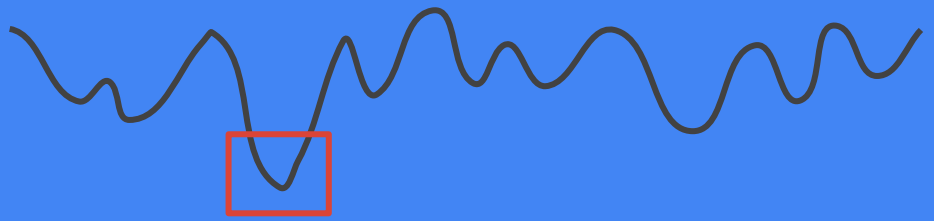
- **monitor** : La métrique à surveiller (ex: val_loss).
- **patience** : Le nombre d'epochs sans amélioration avant d'arrêter l'entraînement.

```
from tensorflow import keras

# Définir le callback EarlyStopping
early_stopping =
keras.callbacks.EarlyStopping(monitor='val_loss',
                                    patience=10)

# Entraîner le modèle avec le callback
model.fit(x_train, y_train,
          epochs=100,
          validation_data=(x_val, y_val),
          callbacks=[early_stopping])
```

ModelCheckpoint



Reprise de l'entraînement : Si votre entraînement est interrompu (panne de courant, erreur, etc.), vous pouvez reprendre l'entraînement à partir du dernier point de sauvegarde.

Meilleur modèle : Vous pouvez configurer ModelCheckpoint pour sauvegarder uniquement le modèle qui obtient les meilleures performances sur un ensemble de validation. Cela vous garantit de conserver le modèle le plus performant, même si les performances se dégradent par la suite.

filepath="meilleur_modele.h5" : Le chemin du fichier où le modèle sera sauvegardé.

monitor="val_accuracy" : La métrique à surveiller pour déterminer le meilleur modèle (ici, l'accuracy sur l'ensemble de validation).

save_best_only=True : Indique de sauvegarder uniquement le modèle qui obtient la meilleure valeur pour la métrique monitor.

mode="max" : Indique que le meilleur modèle est celui qui maximise la métrique monitor. Utilisez mode="min" pour les métriques qui doivent être minimisées (ex: val_loss).

freq_save=10 : Fréquence de sauvegarde en nombre de batchs

ModelCheckpoint

```
# Définir le callback ModelCheckpoint
checkpoint_callback = keras.callbacks.ModelCheckpoint(
    filepath="meilleur_modele.h5",
# Chemin du fichier pour sauvegarder le modèle
    monitor="val_accuracy",  # Métrique à surveiller
    save_best_only=True,
# Sauvegarder uniquement le meilleur modèle
    mode="max"  # Mode de comparaison (max pour l'accuracy)
)

# Entraîner le modèle avec le callback
model.fit(x_train, y_train, epochs=100,
              validation_data=(x_val, y_val),
              callbacks=[checkpoint_callback])
```

```
# Définir le callback ModelCheckpoint
checkpoint_callback = keras.callbacks.ModelCheckpoint(
    filepath="modele_{epoch:02d}_{batch:04d}.h5",
# Format du nom du fichier
    monitor="val_loss",  # Métrique à surveiller (optionnel)
    save_freq=10
# Fréquence de sauvegarde en nombre d'itérations (batchs)
)


# Entraîner le modèle avec le callback
model.fit(x_train, y_train, epochs=100,
              validation_data=(x_val, y_val),
              callbacks=[checkpoint_callback])
```

TensorBoard

TensorBoard est un outil de visualisation pour TensorFlow, Il permet de suivre l'entraînement de vos modèles, d'afficher des graphiques des métriques …

log_dir="logs" : Le répertoire où les logs de TensorBoard seront enregistrés.

%load_ext tensorboard : Charge l'extension TensorBoard dans Jupyter Notebook.

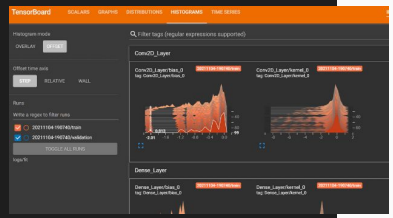
%tensorboard --logdir logs : Lance TensorBoard et affiche les visualisations dans le notebook.

```
tensorboard --logdir logs
```

```
# Définir le callback TensorBoard
tensorboard_callback =
keras.callbacks.TensorBoard(log_dir="logs")

# Entraîner le modèle avec le callback
model.fit(x_train, y_train, epochs=100,
callbacks=[tensorboard_callback])

# Lancer TensorBoard
%load_ext tensorboard
%tensorboard --logdir logs
```

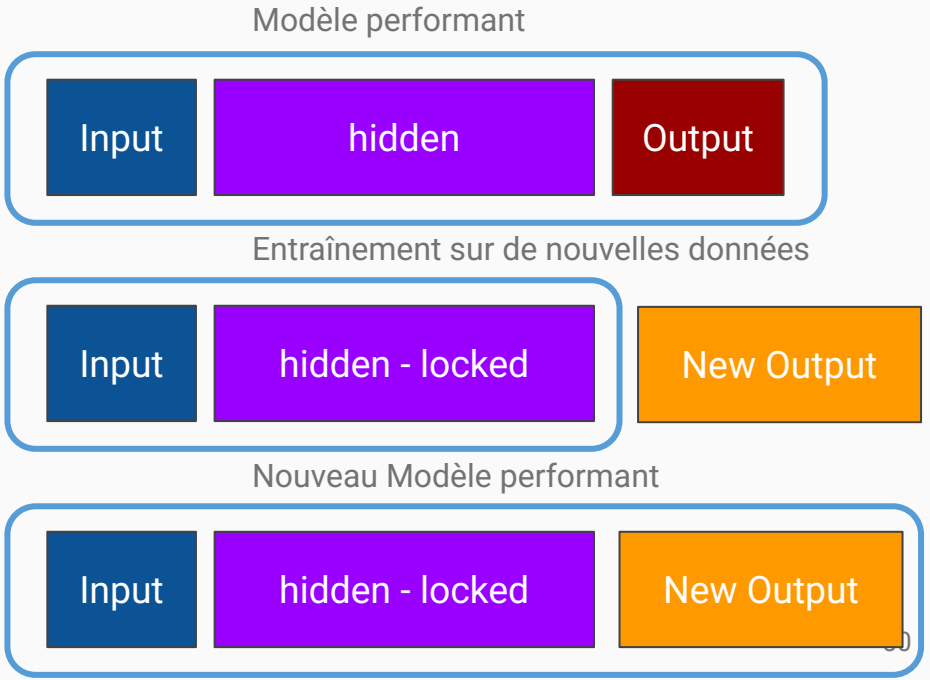


Transfer Learning

Le transfer Learning consiste à utiliser un modèle déjà entraîné pour une tâche similaire et à réentraîner la dernière couche (la tête) sans changer le reste du modèle.

Par exemple un modèle de classification d'images en général, mais on veut le spécialiser sur un problème de classification précis comme reconnaissance de visage.

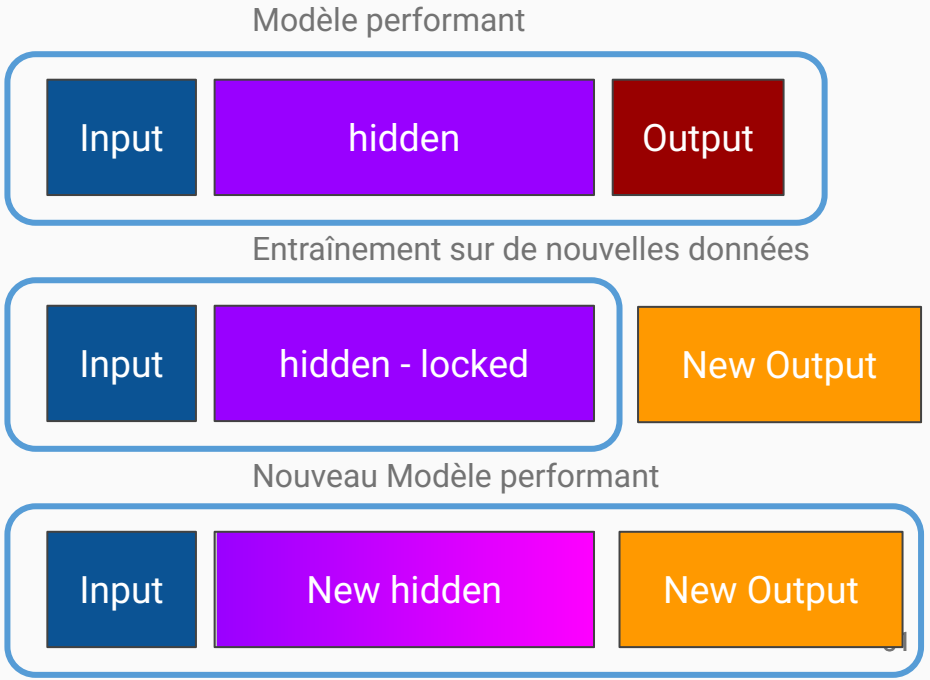
La taille de la sortie est souvent différentes de la taille d'origine.



Fine Tuning

Le Fine Tuning c'est la même chose sauf qu'en plus de mettre une nouvelle tête, on laisse la possibilité à d'autre couche du réseau d'être modifié.

1. on entraîne la nouvelle tête
2. on rend certaine couche entrainable
3. on entraîne le tout une fois de plus



exemples

Transfer learning

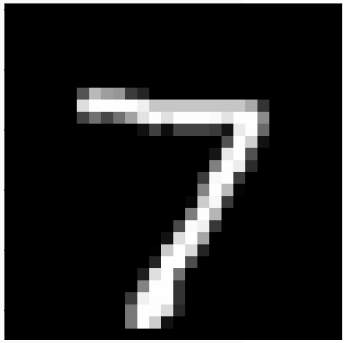
```
def my_model_vgg():
    # load VGG16 model mais on ne garde pas la couche de classification
    model = VGG16(include_top=False, input_shape=(224, 224, 3))
    # on empeche les couches d'être entrainer
    for layer in model.layers:
        layer.trainable = False
    # On utilise le mode fonctionel pour le nouveau modèle
    flat1 = Flatten()(model.layers[-1].output)
    class1 = Dense(128, activation='relu', kernel_initializer='he_uniform')(flat1)
    output = Dense(1, activation='sigmoid')(class1)
    # On crée le modèle
    model = Model(inputs=model.inputs, outputs=output)

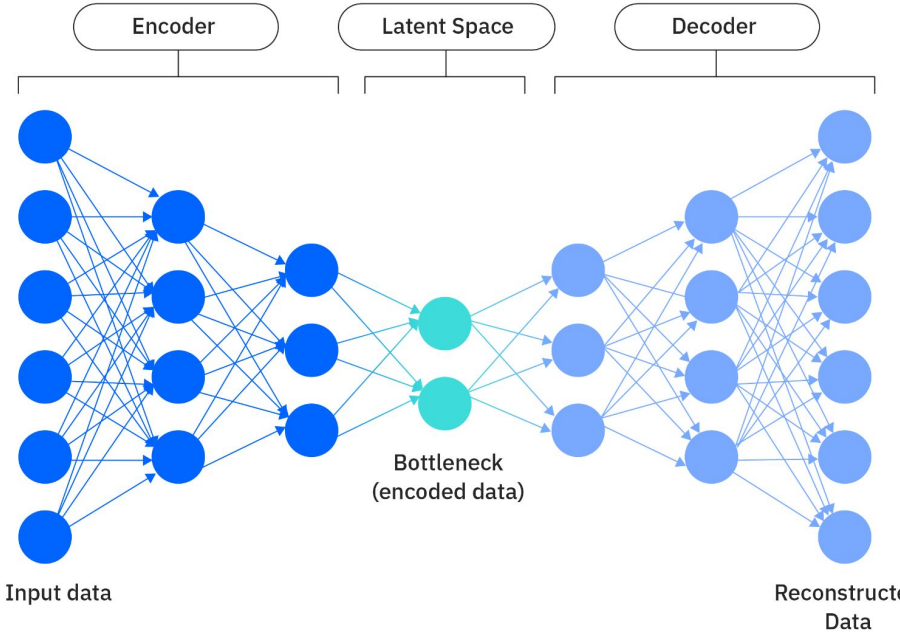
    opt = SGD(lr=0.001, momentum=0.9)
    model.compile(optimizer=opt, loss='binary_crossentropy', metrics=['accuracy'])
    return model
```

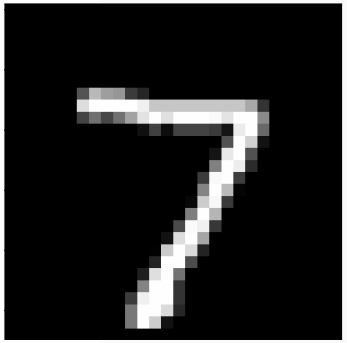
Fine Tuning

```
for layer in model.layers[-4:]:
        layer.trainable = True
```

AUTO ENCODER







AUTO ENCODER

Réduction de dimension (Compression) : Version réseau de neurone d'une PCA

Débruitage (Denoising Auto-encoder) : Image bruité en entré et image propre en sortie

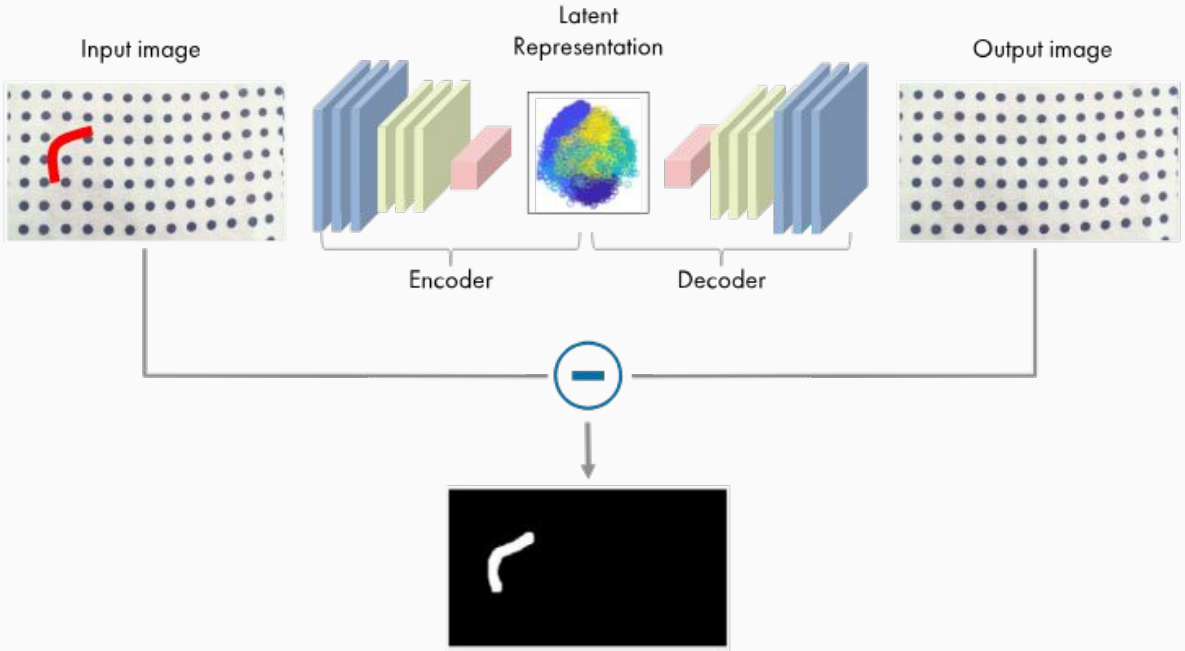
Détection d'anomalies: l'AE ne saura pas "reconstruire" l'anomalie correctement,

Colorisation automatique : image noire et blanc en entré et en sortie image couleur

Super-Resolution : Image basse résolution en entrée et haute résolution en sortie

Génération de données (VAE) : Avec les Variational Auto-encoders on apprend l'espace mathématique des données.

Variational Auto Encoder



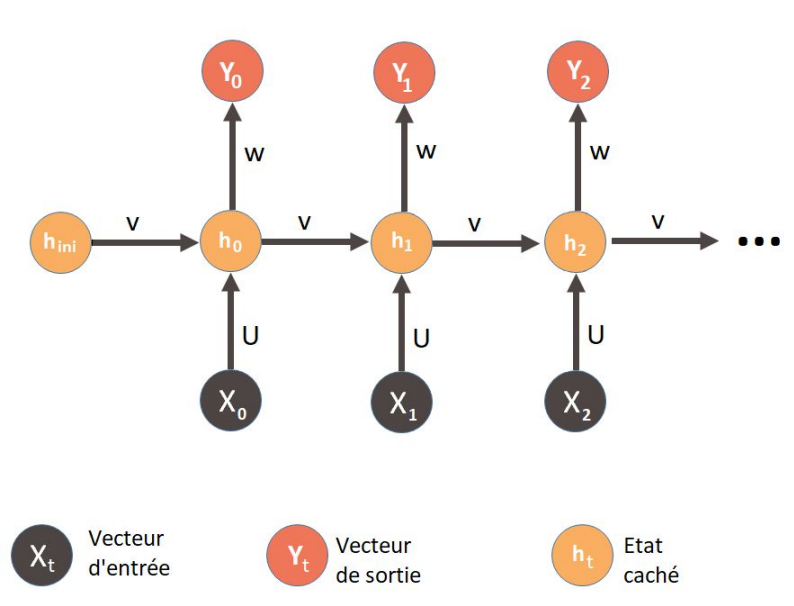
RNN : Recurrent Neural Network

Le problème avec les réseaux de neurones classiques est qu'ils n'ont pas de mémoire.

Chaque ligne d'un dataset est traité indépendamment du passé.

Il n'y a pas d'information qui se propage au cours du temps.

C'est là que les RNN interviennent, ils sont fait pour traiter des séries de données temporelles



En pratique

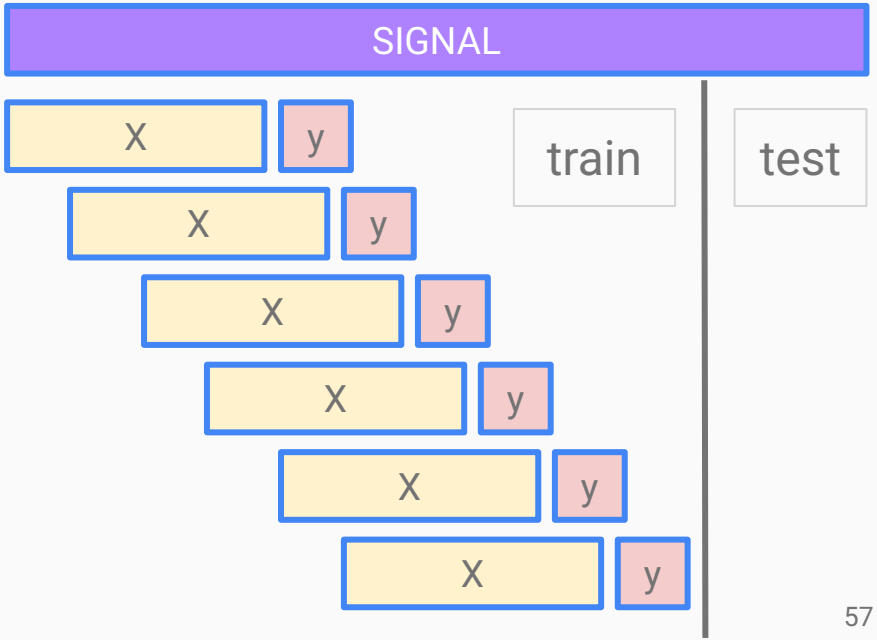
On doit découper le signal en fenêtre glissante.

Sinon:

- Explosion de la RAM
- pas d'apprentissage continu

Piège:

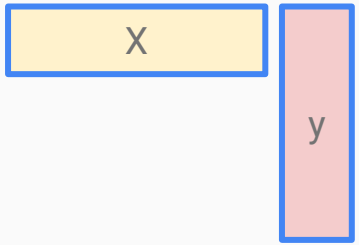
- attention à ne pas prédire le présent
- la taille de "y" définit l'horizon de prédiction (10 x 1h à la place de 1 x 1h)

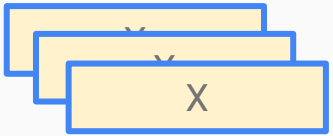


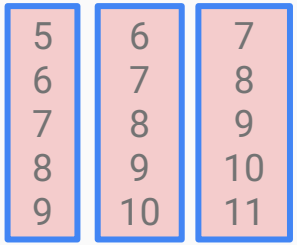
Préparation des données pour prédire plusieurs valeurs

pour une valeur

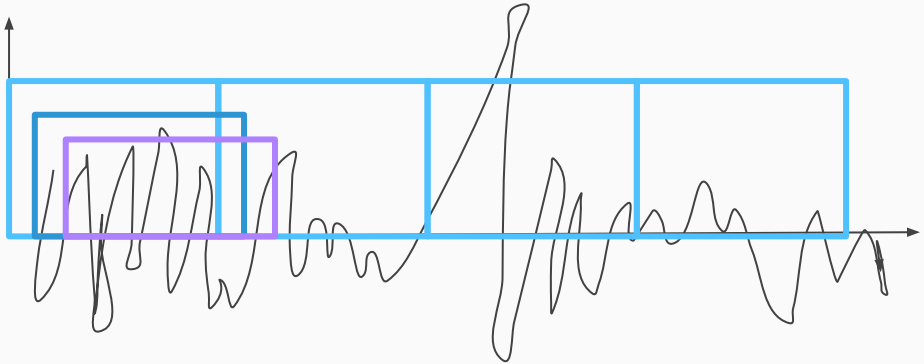






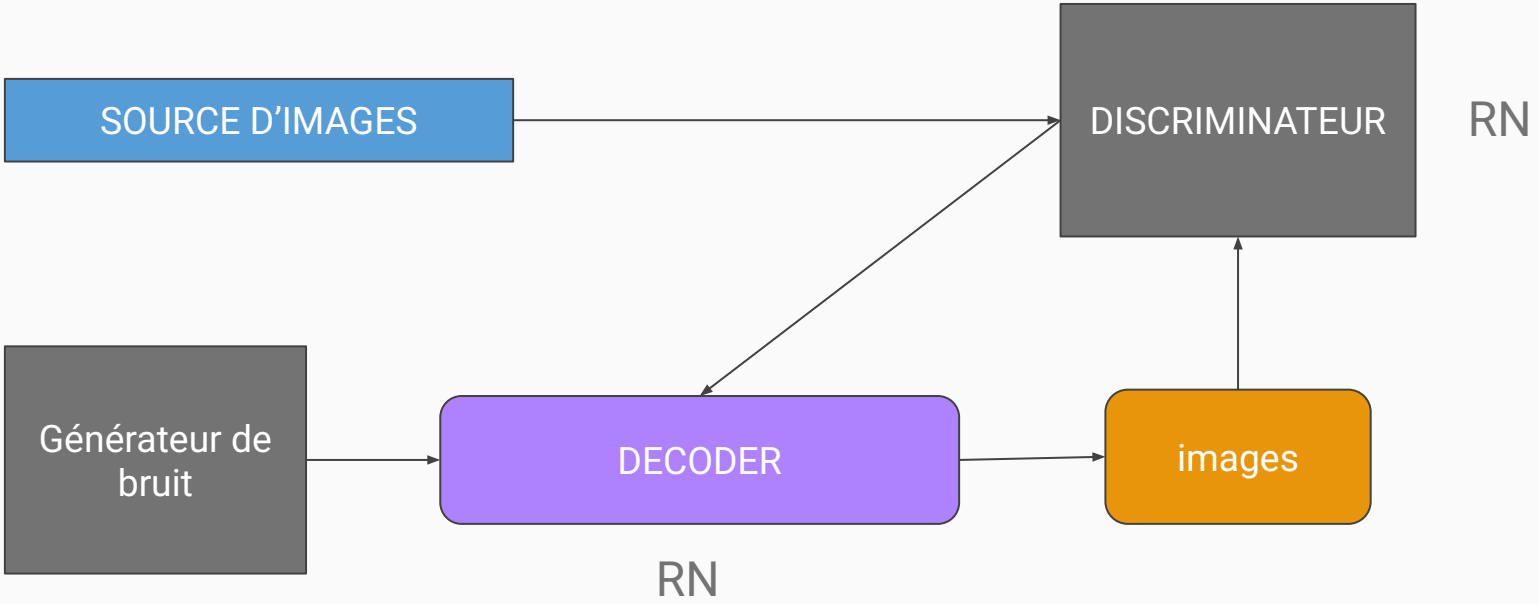



```
`Y_train = np.lib.stride_tricks.sliding_window_view(train[WINDOW:], window_shape=HORIZON)`
```

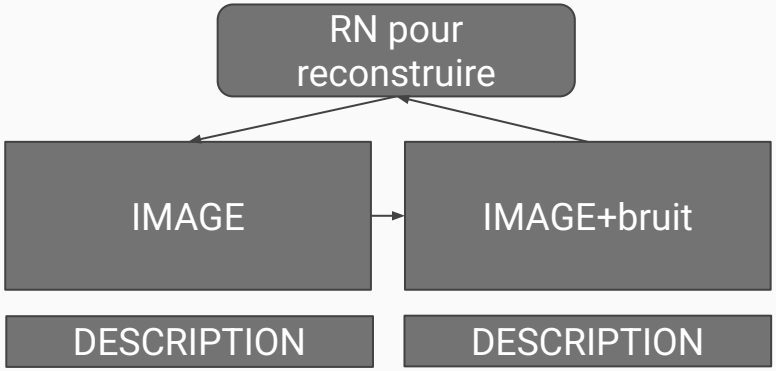


```
train_dataset =
tf.keras.utils.timeseries_dataset_from_array(
    data=train,
    targets=train[WINDOW:],
    sequence_length=WINDOW,
    sequence_stride=1,
    batch_size=BATCH_SIZE,
    shuffle=False
)
```

GAN - Generative adversarial Network



Diffusion



PROMPT

```
for n in range(10):
    print(n)
```

```
for n=0;n<10;n++
{
print(n)
}
```

```
pour n allant de 0 à 10 par pas de 1 faire:
    afficher(n)
```

